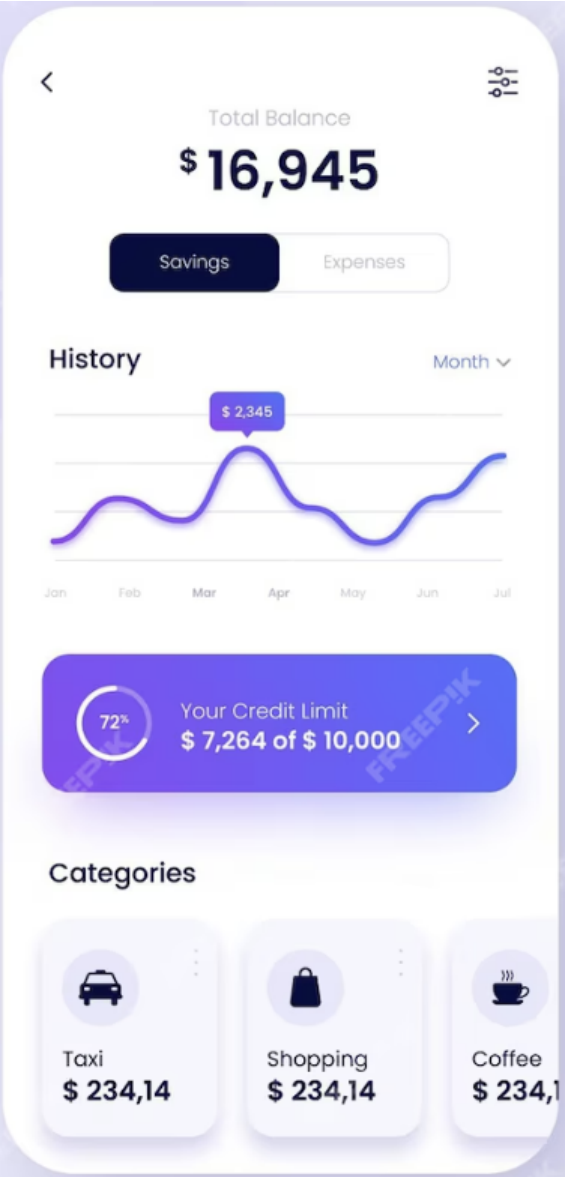
Design Patterns

Dr. Adnan Arefeen

Dept. of ECE
North South University

An app interface



What Are Design Patterns?

- Design patterns are typical solutions to commonly occurring problems in software design.
- They are not copy-paste code but **conceptual templates** adaptable to specific contexts.
- Design patterns are a toolkit of tried and tested solutions to common problems in software design.
- Unlike algorithms (which define exact steps), patterns describe **high-level solutions**.
- **Analogy:**
  - Algorithm = *Recipe (precise steps)*
  - Pattern = *Blueprint (flexible structure)*

A scenario

Scenario: You're developing a large Java application with multiple modules (auth, analytics, billing).
Problem: Each module creates its own `Logger` instance.

A scenario

Scenario: You're developing a large Java application with multiple modules (auth, analytics, billing).
Problem: Each module creates its own `Logger` instance.

- Memory waste due to multiple objects
- Inconsistent logging behavior
- No centralized control

Why is this a Problem?

- Redundant instantiation
- Harder to maintain and debug logs
- Difficult to configure logging globally
- Violation of Single Responsibility Principle (each module managing its own logger)

Step 1: Define the Goal

We need:

- One single shared `Logger` object
- Accessible from any part of the application
- Globally consistent formatting
- Efficient use of memory

Step 1: Define the Goal

We need:

- One single shared `Logger` object
- Accessible from any part of the application
- Globally consistent formatting
- Efficient use of memory

Solution? Use the Singleton Pattern

Step 2: What is Singleton?

Definition: Ensure a class has only **one instance**, and provide a global point of access to it.

- Private constructor
- Static instance variable
- Static method to return the instance

Step 3: Logger Singleton Implementation

```java
public class Logger {
    private static Logger instance;
    private Logger() {
        System.out.println("Logger initialized");
    }
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }
    public void log(String msg) {
        System.out.println("LOG: " + msg);
    }
}
```

Step 4: Use it in Your Application

```java
public class Main {
    public static void main(String[] args) {
        Logger log1 = Logger.getInstance();
        Logger log2 = Logger.getInstance();
        log1.log("User login attempt");
        log2.log("Payment processed");
        System.out.println(log1 == log2); // true
    }
}
```

Step 4: Use it in Your Application

```java
public class Main {
    public static void main(String[] args) {
        Logger log1 = Logger.getInstance();
        Logger log2 = Logger.getInstance();
        log1.log("User login attempt");
        log2.log("Payment processed");
        System.out.println(log1 == log2); // true
    }
}
```

Output: Single instance shared across calls

Step 5: What Did We Solve?

- Reduced object creation
- Centralized logging configuration
- Consistent behavior across modules
- Easy maintenance and testing

Step 5: What Did We Solve?

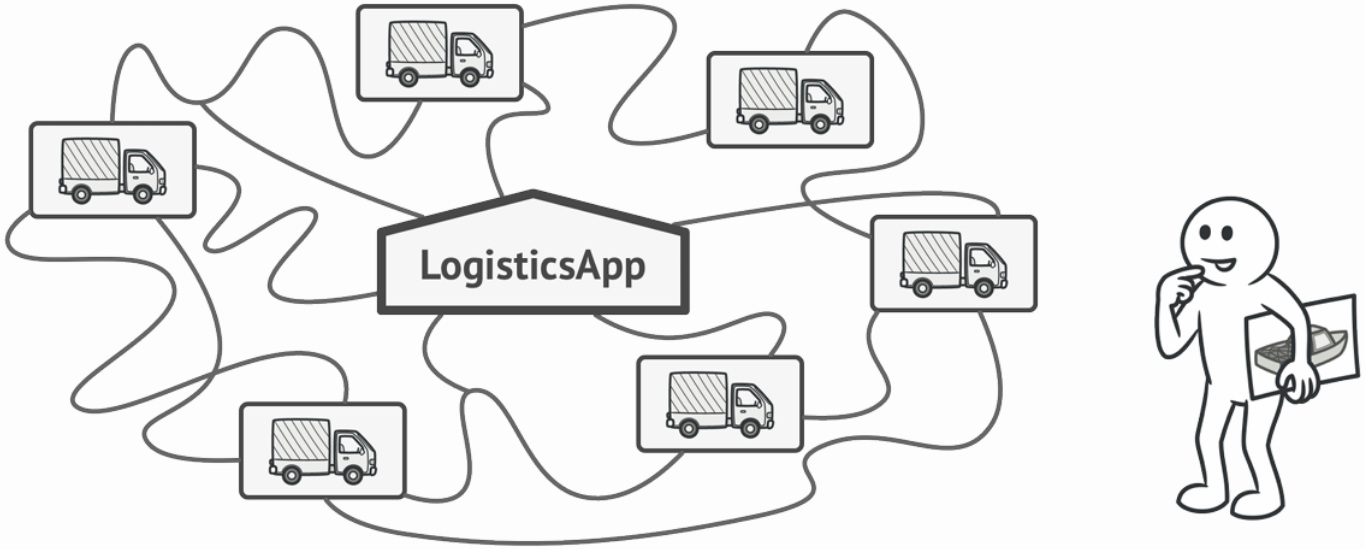
- Reduced object creation
- Centralized logging configuration
- Consistent behavior across modules
- Easy maintenance and testing

Singleton pattern gives us control over instantiation.

Problem Scenario

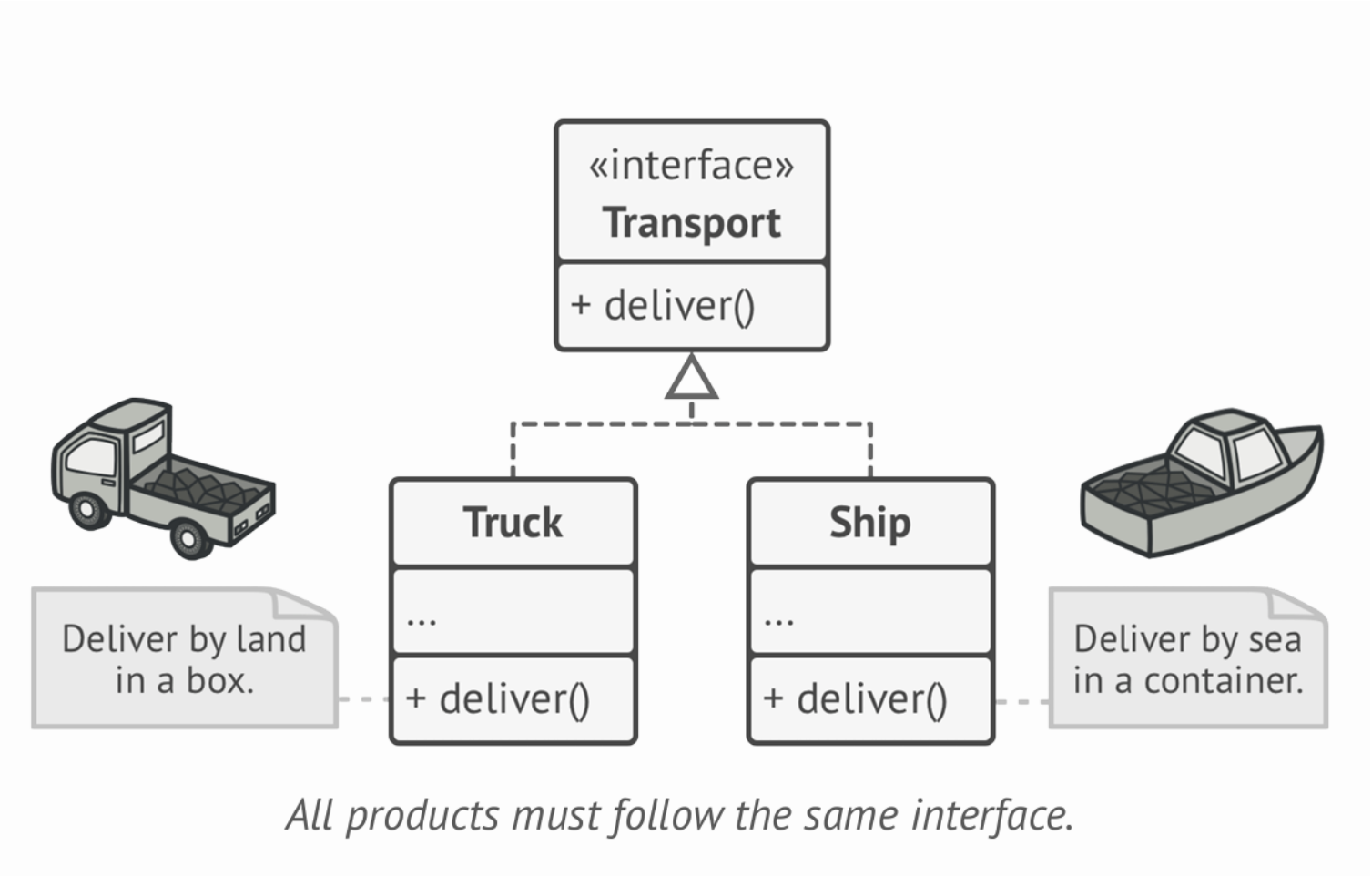
1. Imagine that you're creating a logistics management application. The first version of your app can only handle transportation by trucks, so the bulk of your code lives inside the **Truck** class.
2. After a while, your app becomes pretty popular. Each day you receive dozens of requests from sea transportation companies to incorporate sea logistics into the app.

Problem Scenario



Adding a new class to the program isn't that simple if the rest of the code is already coupled to existing classes.

Factory Method Pattern



Problem with this design?

```
public class Main {
    public static void main(String[] args) {
        String type = "A";
        Product product;

        if (type.equals("A")) {
            product = new ConcreteProductA();
        } else if (type.equals("B")) {
            product = new ConcreteProductB();
        } else {
            throw new IllegalArgumentException("Unknown type
    ");
        }

        product.doStuff();
    }
}
```

Problem with this design?

Issues:

- Tight coupling to concrete classes
- Difficult to extend with new types
- Violates Open/Closed Principle

Step 1: Define Product Interface and Implementations

```java
public interface Product {
    void doStuff();
}

public class ConcreteProductA implements Product {
    public void doStuff() {
        System.out.println("Doing work in Product A");
    }
}

public class ConcreteProductB implements Product {
    public void doStuff() {
        System.out.println("Doing work in Product B");
    }
}
```

Step 2: Define Creator and Subclasses

```
public abstract class Creator {
    public void someOperation() {
        Product product = createProduct();
        product.doStuff();
    }

    public abstract Product createProduct();
}

public class ConcreteCreatorA extends Creator {
    public Product createProduct() {
        return new ConcreteProductA();
    }
}

public class ConcreteCreatorB extends Creator {
    public Product createProduct() {
        return new ConcreteProductB();
    }
}
```

Step 3: Refactored Main Class

```java
public class Main {
    public static void main(String[] args) {
        Creator creator;
        String type = "A";

        if (type.equals("A")) {
            creator = new ConcreteCreatorA();
        } else if (type.equals("B")) {
            creator = new ConcreteCreatorB();
        } else {
            throw new IllegalArgumentException("Unknown type");
        }

        creator.someOperation();
    }
}
```

Benefits of Refactoring with Factory Method

- Separation of creation logic from usage logic
- Easy to add new product types without modifying client code
- Follows Open/Closed Principle
- Promotes clean architecture and testing

Why Factory Method May Feel "Messy" at First

Downsides You Might Notice:

- **More classes:**
  Creator, ConcreteCreatorA, ConcreteProductA, etc. feels like overkill in simple cases.
- **Extra layers of indirection:**
  Instead of directly calling `new ConcreteProductA()`, you go through a factory method.
- **Harder to trace the logic:**
  For beginners, it's unclear "where is the object really being created?"